Notion ReAct Planner Agent

Duration: 8–9 Hours

Introduction

The **Notion ReAct Planner Agent** is a smart, reasoning-driven daily planning system powered by a **ReAct (Reason + Act) AI agent**, deeply integrated with **Notion**.

Unlike traditional task managers that only store information, this system actively **thinks through user intent, priorities, schedules, and real-world context** before taking action. Users can interact with the agent in natural language, similar to speaking with a personal assistant, and the agent decides which tools to use—such as fetching tasks, updating notes, scheduling calendar events, or checking external context like weather.

The core objective is to transform **Notion from a passive workspace into an active, intelligent planning assistant** that reasons before acting, helping users plan their day more effectively and context-aware.

Aim of the Project

To build a **reasoning-driven daily planning system** using a ReAct-based AI agent that understands user intent, plans appropriate actions, and actively manages tasks and calendar events inside Notion.

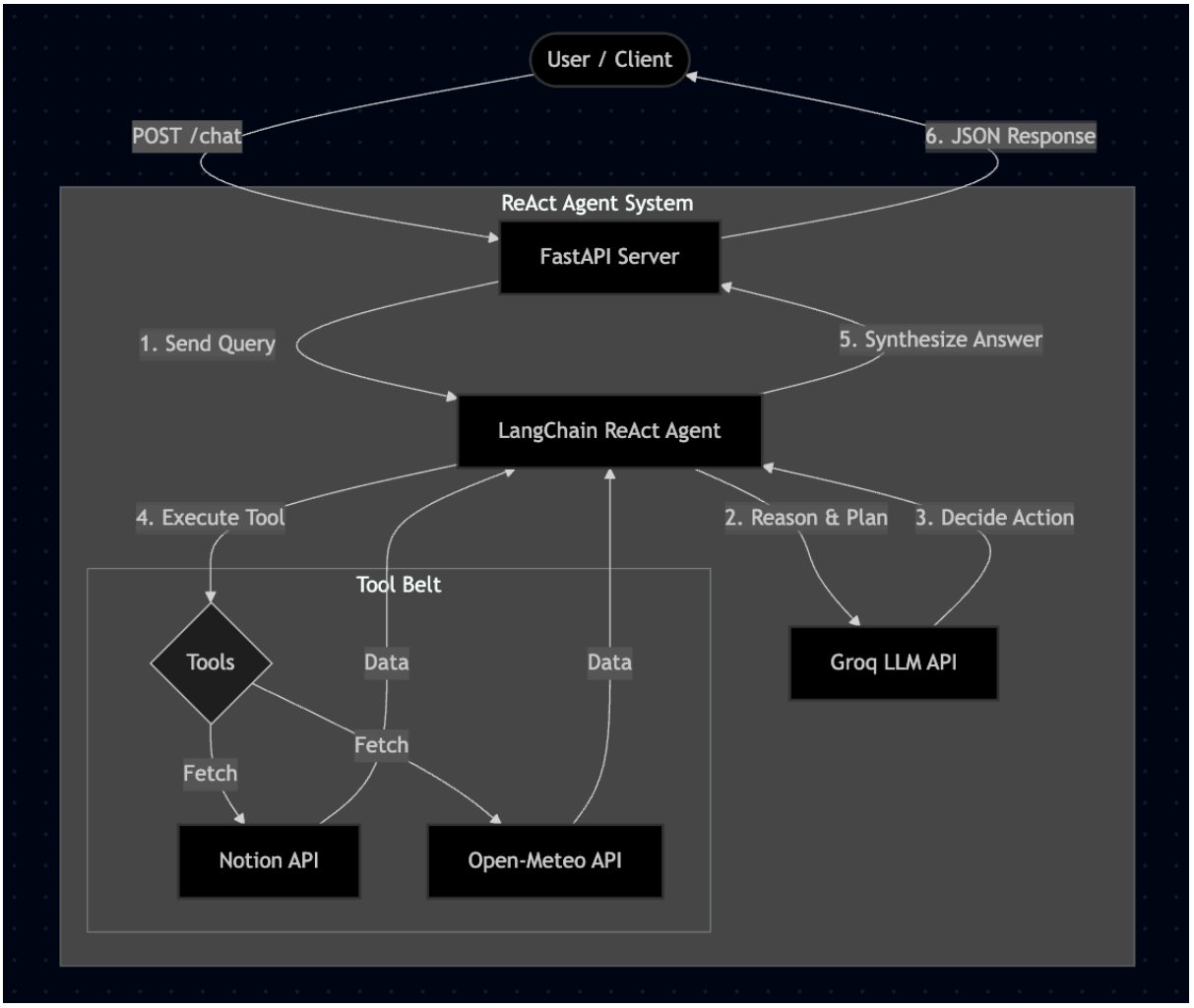
Dataset Used

Not Applicable (NA)
This project operates in real time using live user inputs, Notion workspace data, calendar events, and external APIs instead of static datasets.

Tools & Technologies Used

- **Python 3.12+**
- **FastAPI**
- **LangChain**
- **Groq API (LLM Inference)**
- **Notion API**
- **Open-Meteo API**
- **Uvicorn**
- **Requests**

Architecture Diagram



Key Modules

1. Main Entry Module (`main.py`)

Bootstraps the application, loads environment variables, initializes configurations, and starts the Uvicorn server to run the application.

2. API Gateway Layer (`api/server.py`)

A FastAPI-based server responsible for handling HTTP requests, routing, request validation, and communication between the frontend and the AI agent.

3. Agent Intelligence Core (`agent/bot.py`)

Acts as the brain of the system. It initializes the ReAct agent, manages the reasoning loop, interprets user intent, and orchestrates tool usage through the LLM.

4. Notion Notes Integration (`tools/notion_notes.py`)

Handles interaction with the Notion API to fetch, read, and create notes dynamically based on agent decisions.

5. Calendar Management Module (`tools/notion_calendar.py`)

Manages scheduling logic, calendar event creation, retrieval, and updates within the Notion Calendar.

6. Weather Service Interface (`tools/weather.py`)

Integrates with the Open-Meteo API to fetch real-time weather data, enabling context-aware planning decisions.

7. Utility & Configuration Layer (`utils/`, `.env`)

Manages shared helper functions, logging, error handling, and secure environment variable configuration.

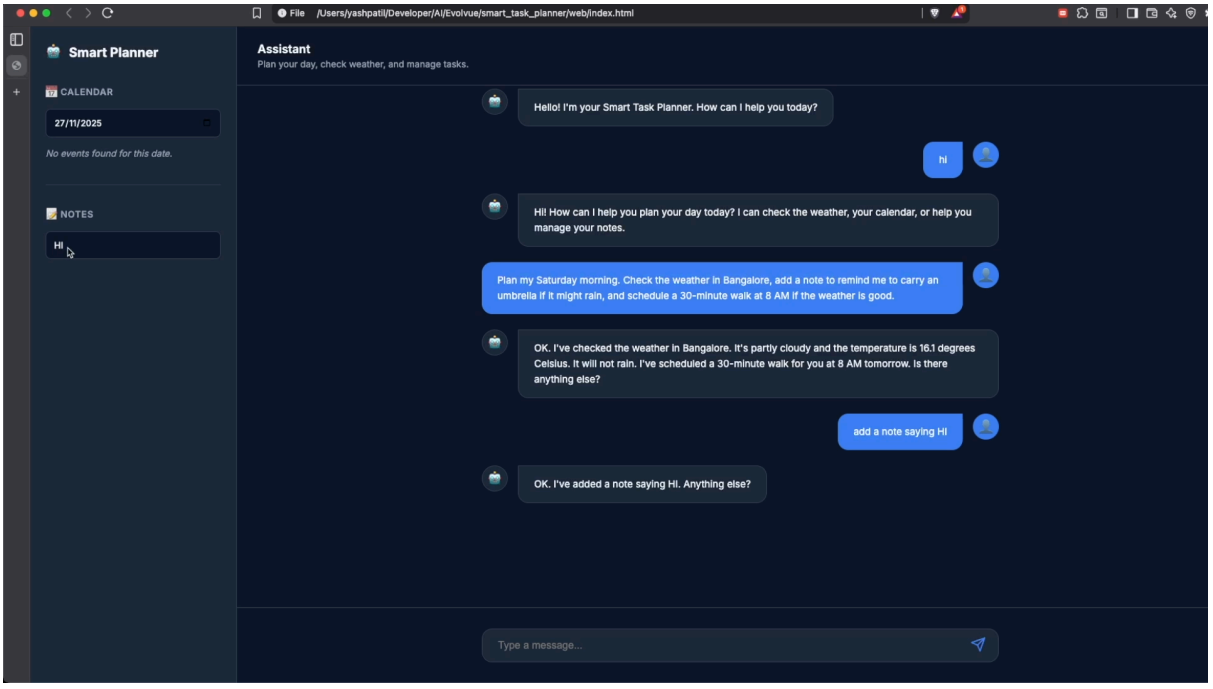
8. Static Frontend (`static/`)

Serves a lightweight client-side interface (HTML, CSS, JavaScript) that allows users to interact with the agent through a browser.

Learning Outcomes

- Mastering the **ReAct Pattern** for AI reasoning
- Building **tool-using AI agents**
- Designing **multi-tool agent pipelines**
- Structuring **production-ready AI applications**
- Deploying AI agents to **production environments (AWS)**

Screenshots



Optional Add-ons

1. Add **pytest** for CI-level automated testing
2. Implement an **authentication layer** between client and server
3. Improve and refine **agent prompts** for better reasoning accuracy
4. Integrate additional tools such as:
    - Web Search Tool
    - Email Management Tool
    - Calculator Tool

Resource links

1. LangChain Agents Documentation
   https://docs.langchain.com/oss/javascript/langchain/agents
2. Notion API Documentation
   https://developers.notion.com/